number as a parameter and returns an array containing two objects of the same class, whose values are equivalent to its parameter and itself.

For our Roman number class, it's fairly easy. Internally, each Roman number object holds its real value as an Integer in an instance variable, @value. The coerce method checks to see whether the class of its parameter is also an Integer. If so, it returns its parameter and its internal value. If not, it first converts both to floating point.

```
class Roman
  def initialize(value)
    @value = value
  end

  def coerce(other)
    if  Integer === other
      [other, @value]
    else
      [Float(other), Float(@value)]
    end
  end

  # .. other Roman stuff
end

iv = Roman.new(4)
xi = Roman.new(11)

3 * iv    # => 12
1.2 * xi  # => 13.2
```

In the last two lines, the numeric left side of the argument calls coerce on the Roman numeral, receives the coerced array of values, and multiplies them.

Class Roman as implemented doesn't know how to do addition. You couldn't have written xi + 3 in the previous example, because Roman doesn't have a + method. Let's go wild and implement addition for Roman numbers. The whole class might look like this:

ducktyping/roman3.rb

```
class Roman
  MAX_ROMAN = 4999

  attr_reader :value
  protected :value

  def initialize(value)
    if value <= 0 || value > MAX_ROMAN
      fail "Roman values must be > 0 and <= #{MAX_ROMAN}"
    end
    @value = value
  end

  def coerce(other)
    if Integer === other
      [other, @value]
    else
      [Float(other), Float(@value)]
    end
  end
```